

Software Architecture and Design

01- Introduction

We don't just write code. We design systems.

Objectives

- 1 Define what *software architecture* means
- 2 Recognize why large systems need structure
- 3 Identify components in real-world systems
- 4 Get introduced to UML as a design tool

 Think: What does “architecture” mean to you?

What is Architecture?

- **Architecture = Structure + Organization + Plan**
 - Used in buildings, cities, systems
 - Defines *how parts fit together*
 - Acts as a **blueprint**



Question: Can you think of different types of architecture?

What is Architecture?

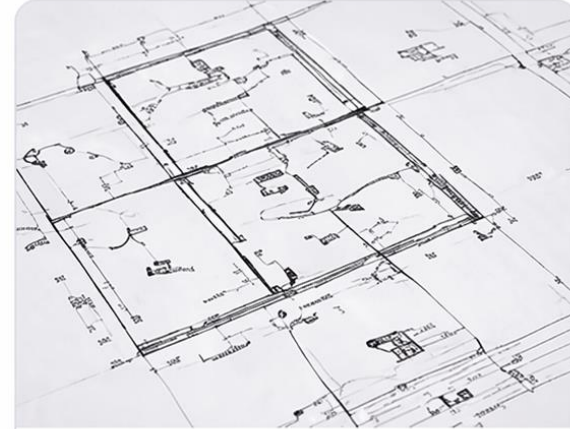
The word **architecture** can have many meanings depending on the context.



Art and science of designing spaces (**structures, buildings, cities, parks, landscapes...**)



A plan design of any kind of **system** (Software system, Network system, Ship)



Architecture = Blueprint = Outline

From Real World to Software

- **Architecture exists everywhere:**

-  Buildings → structural design
-  Electrical systems → circuit design
-  Software → system structure
-  Same idea, different domains

 Question: Which one is closest to what we study?

When Do We Need Architecture?

- **Not all systems need heavy design**
 - Small program → simple structure
 - Large system → complex design needed
 - 👉 Complexity decides the need



Think!

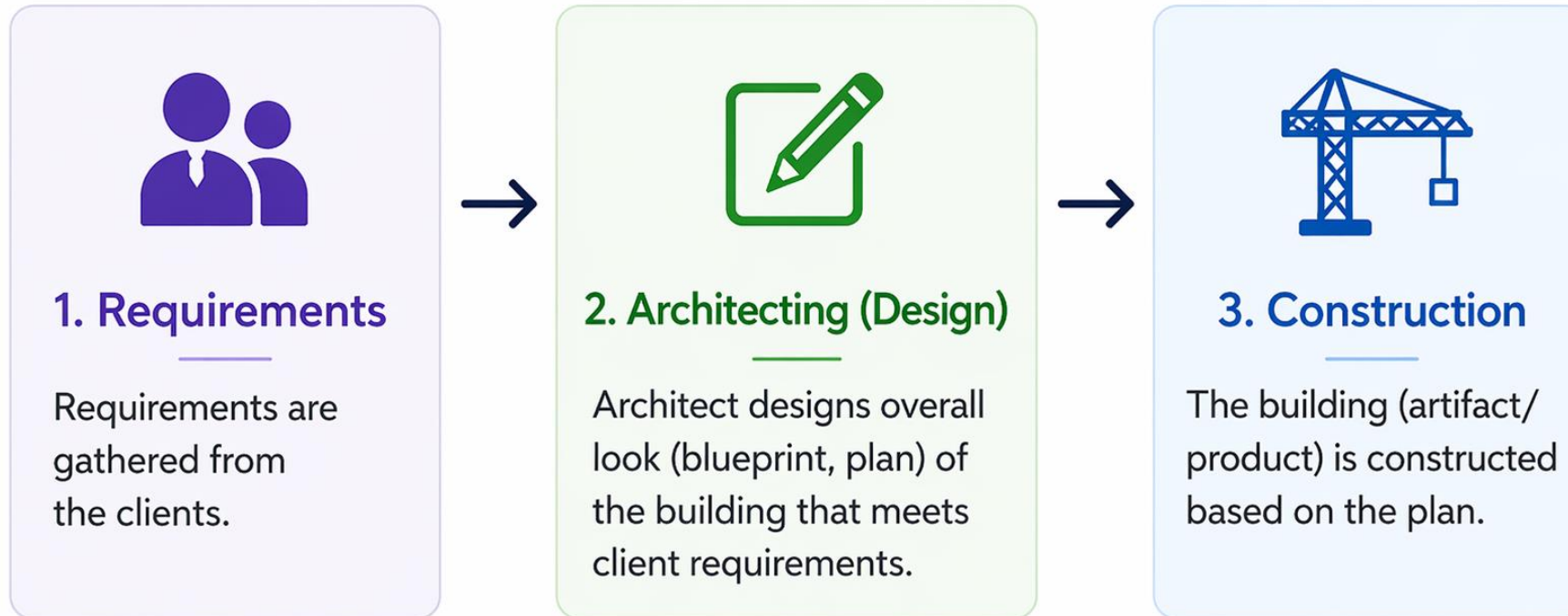
Is it necessary to have an architect for every small thing we build?
Why or why not?



Poll: When is architecture critical?

A. Small apps B. Medium systems C. Large / scalable systems D. Always

Consider a process of constructing a house



Interactive Question: What happens if we start building without a plan?

Process of constructing a building structure-output

Plans may include drawings of:

- ✓ Structural system
- ✓ Air-conditioning, heating, ventilating systems
- ✓ Electrical systems
- ✓ Communications systems
- ✓ Plumbing
- ✓ Landscape plans
- ✓ Building materials
- ✓ Interior furnishings and more



Constraints to consider:

- Budget
- Time
- Space
- Regulations
- Resources
- ...and more



Interactive Question: Which constraint is usually the hardest to manage?

Architectural Process



Architects are not concerned with the creation of building technologies, making materials, or making glasses with better thermal qualities or stronger concrete.

Architects are concerned with how building materials can be put together in desirable ways to complete a building suited to its purpose.

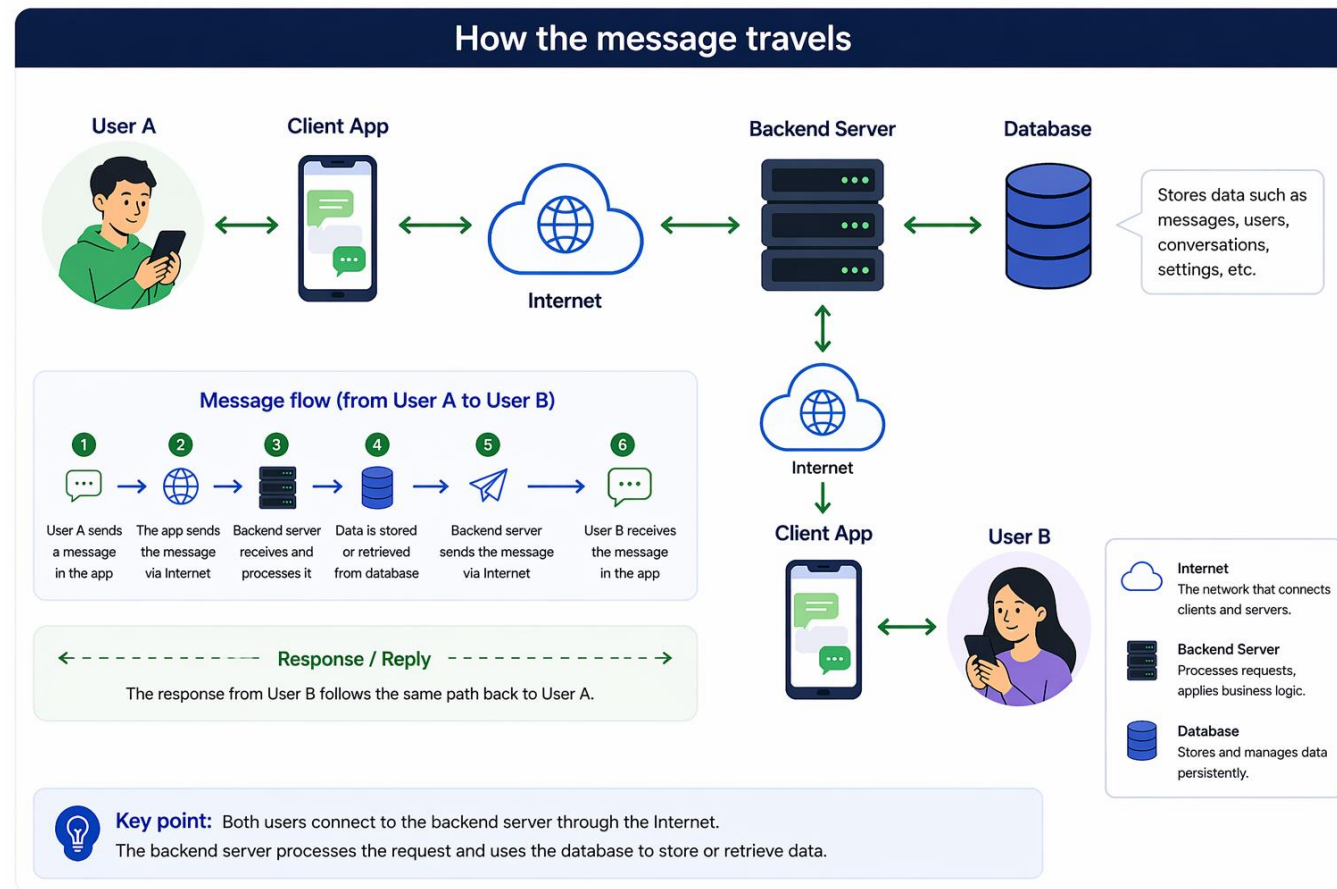
The design process that assembles the parts in a useful way, that **satisfies client needs in a cost-effective way (sometimes)**, is called “architecture.”

💬 Question: 🙋 In software, what are the “materials” we work with?

Think About This

- **How does a message travel in a system like:**
 - WhatsApp
 - Instagram
 - Food delivery apps
 - 👉 Is it just one program?
- 💬 **Discuss (2 mins):**
 - Where is data stored?
 - How does it reach another user?

Think About This



Systems are NOT One Thing

- **A system = collection of components**
 - Typical structure:
 - User Interface
 - Backend Logic
 - Database
 - Network
 - 👉 Systems are **distributed and connected**

💬 Question: What happens if one part fails?

Basic Architecture Concept

- **Software Architecture =**
 - Components
 - Interactions
 - Constraints
 - 👉 Focus = *big picture*, not code

💡 Question: Why not build everything in one file?

Sri Lankan Example: Bus Booking System

- **System Features**
 - Search buses
 - Book seats
 - Online payment
- **Main Components**
 - User App
 - Booking Service
 - Payment Gateway
 - Database



Think: What can go wrong in this system?

Sri Lankan Example: Food Delivery System

- **Multiple Actors**
 - Customer
 - Restaurant
 - Delivery Rider
- **System Must Handle**
 - Orders
 - Tracking
 - Notifications



Question: How do these actors communicate?

Who is a Software Architect?



- **A Software Architect**

- Designs the overall system
- Makes high-level decisions
- Ensures system works as a whole

- **Skills Needed**

- Technical knowledge
- Problem solving
- System thinking
- Communication
- Leadership and Management
- Artistic Skill

💬 Question: What skill do you think is most important?

Who is a Software Architect?



- **A software architect is responsible for:**

- Designing the overall structure of a software system
- Making high-level design decisions
- Selecting technologies and frameworks
- Guiding the development team
- Ensuring quality attributes (performance, security, scalability, etc.)

💬 Question: What do you think is the biggest challenge for a software architect?

Architecture vs Design

Architecture	Design
Big picture	Details
System structure	Component logic
Long-term decisions	Implementation

- 👉 Architecture comes first

💬 Question: Can we design without architecture?

Why Do We Need UML?

- **Problem:**
 - Hard to explain systems verbally
- **Solution:**
 - 👉 UML (Unified Modeling Language)
- **Used for:**
 - Visualizing systems
 - Communicating ideas
 - Designing clearly

💬 Question: Why are diagrams better than text?

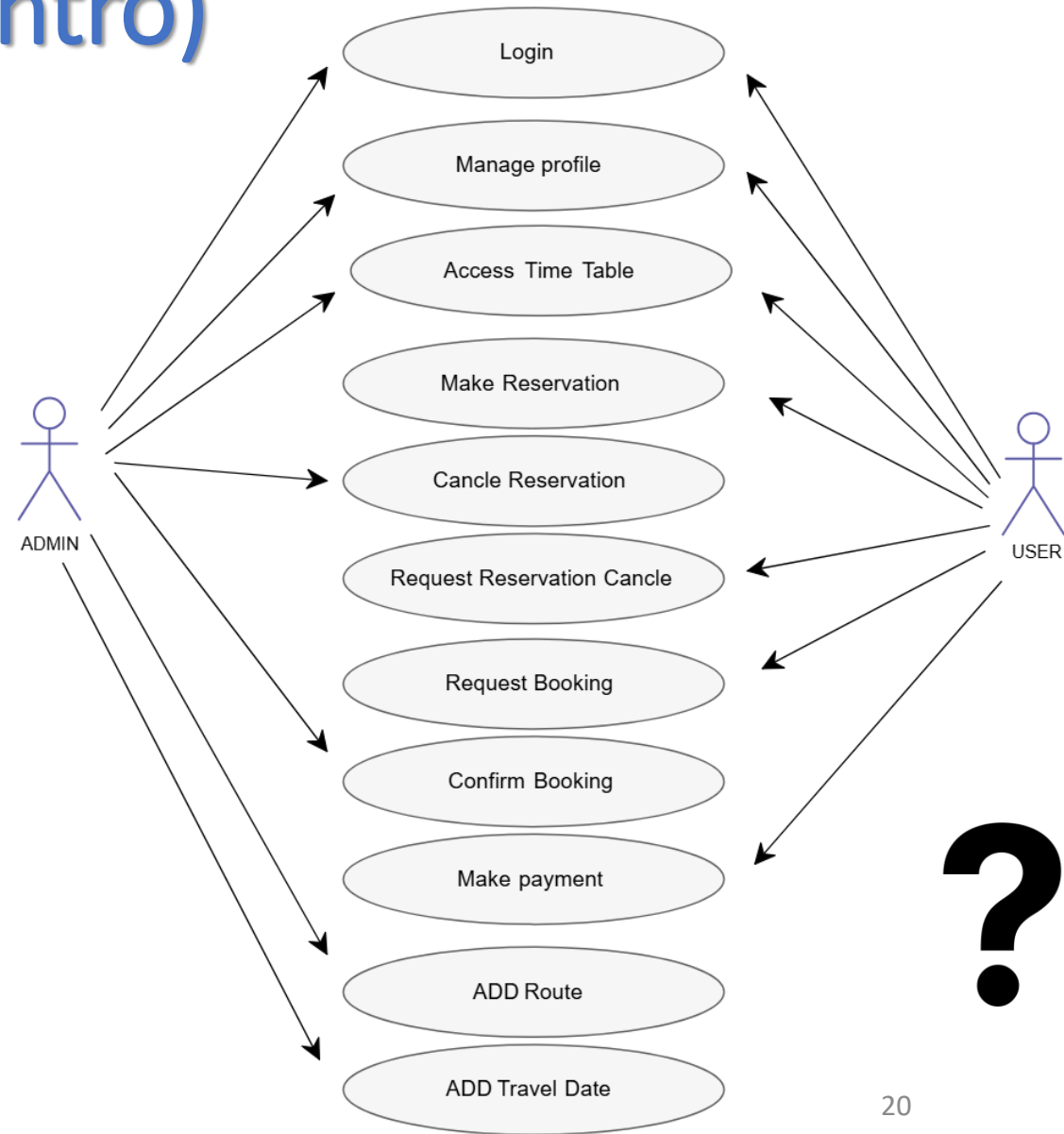
UML: Use Case Diagram (Intro)

- **Shows:**

- What the system does
- Who interacts with it
- Example:
 - User → Book seat
 - User → Make payment

- **Question:**

Who are the actors in our system?



UML: Component Thinking

- **Break system into:**
 - Modules
 - Services
 - Layers
 - 👉 Each component has a responsibility

💡 Question: Which component is most critical?

Quality Attributes (Intro)

- A good system is not just functional:
 - ⚡ Performance
 - 🔒 Security
 - 📈 Scalability
 - 🛠️ Maintainability
- 👉 Architecture decides these

💬 Question: Which matters most for a bank system?

Mini Activity

- Design a system for:
👉 *Online Food Ordering*
- Include:
 - Components
 - Interactions
 - (Optional) simple UML
- 👥 Work in groups of 3–4

Why Study Software Architecture?



Build systems
that are scalable
and maintainable

Design systems that
can grow with your
business and are easy
to maintain.



**Increase
quality**

Improve quality,
security and
reliability to deliver
better experiences
and reduce defects.



**Reduce cost
and time**

Build efficiently
to reduce cost
and time in the
long run.



**Better
communication**

Improve communication
with teams &
stakeholders for
better alignment and
decision-making.



**Stronger career
opportunities**

Build in-demand skills
and experience to
unlock growth and
opportunities in the
industry.

 *“Anyone can write code.
Designing systems is what makes you an engineer.”*